

BlurGuard – Real-Time Screen-Share Sensitive Data Censor

Project Proposal for UCS503P

Submitted to: Nisha Thakur

Thapar Institute of Engineering and Technology

Project Team Members: Lakshita Dabra, Yashjot, Rhythm

August 16, 2026

1 Higher-order goal

...secondary framing

This project aims to improve privacy during digital communication and screen sharing by reducing the risk of unintentionally exposing sensitive information. While the topic is broader than any single application, the immediate engineering objective is to translate *privacy-preserving screen sharing* into a measurable, deployable software solution.

2 Time-to-value

...primary heuristic

- We will deliver meaningful increments quickly by prototyping the core screen-capture → OCR → detection → masking workflow and validating it with representative test screens.
- We will prioritize fast feedback over perfection by using short iterations and introducing automated testing and CI early to reduce integration and release friction.
- Success is defined by what can be detected, censored, and measured in the first iteration, then improved using evaluation results and user feedback.

3 Problem Statement

During online meetings, demonstrations, remote assistance, and screen sharing, users may accidentally expose sensitive information such as phone numbers, email addresses, government identifiers, passwords, API keys, account details, or confidential documents. Common pain points include:

- **Accidental exposure:** sensitive text can appear on screen for a few seconds before a user notices it.
- **Manual protection:** users must remember to close, hide, or blur private information before sharing a screen.
- **Context switching:** sensitive information can appear in different applications, windows, documents, or browser pages during a live session.
- **Privacy risk:** screen content may contain information that should not be stored or transmitted outside the user's device.

4 Proposed Solution

...Software Proposition

4.1 Overview

Build a privacy-focused real-time “BlurGuard” system with the following components:

- **Screen capture module:** capture the shared screen or selected region at a controlled frame rate.
- **OCR/text extraction:** convert visible text into machine-readable text with bounding-box coordinates.
- **Sensitive-data detection:** identify PII and secrets using regular expressions, rule-based checks, and lightweight AI/ML or NLP assistance.
- **Real-time censoring:** automatically blur, mask, or replace detected sensitive regions before protected output is presented.
- **User dashboard:** allow users to configure protected data types, review detections, and monitor system activity.
- **Privacy/security layer:** prefer local processing, minimize storage, and avoid retaining raw screen content by default.

4.2 Core workflow

1. User starts BlurGuard and selects the screen, window, or region to monitor.
2. The system captures frames and performs preprocessing to improve OCR quality.
3. OCR extracts visible text and its screen coordinates.
4. Detection rules and AI/ML-assisted classification identify possible sensitive information.
5. Sensitive regions are assigned a risk level and blurred/masked in real time.
6. The protected screen output is used for screen sharing while raw sensitive content remains on the user’s device.

4.3 Operational constraints for an educational, privacy-focused setting

- Keep processing lightweight enough for common laptops and mid-range systems through controlled capture rates and efficient OCR.
- Avoid dependence on advanced research algorithms; focus on workflow, detection correctness, latency, and privacy-preserving engineering.
- Prefer local-first processing so screen frames and extracted text are not unnecessarily sent to external services.
- Design the prototype so components can be replaced or extended without redesigning the complete system.

5 Solution Approach

...Engineering focus

This is an engineering course project, so the focus is on a practical real-time privacy pipeline rather than claiming novel OCR or machine-learning research.

5.1 Sensitive-data detection

...non-research

- Normalize OCR text and use pattern matching for structured information such as email addresses, phone numbers, URLs, identifiers, and common secret formats.

- Use lightweight NLP/AI-ML classification where contextual information is needed to distinguish sensitive text from ordinary text.
- Associate detected text with OCR bounding boxes so the exact on-screen region can be censored.
- Use confidence/risk thresholds and allow user-configurable rules rather than automatically assuming every detection is sensitive.
- Keep uncertain detections reviewable and avoid storing raw screen content unless explicitly required for testing.

5.2 Real-time processing architecture

...privacy-first pipeline

- **Capture layer:** MSS/PyAutoGUI or an equivalent screen-capture mechanism.
- **Preprocessing layer:** OpenCV operations such as resizing, grayscale conversion, thresholding, and region selection.
- **OCR layer:** Tesseract or EasyOCR for text extraction and bounding boxes.
- **Intelligence layer:** regex/rules + NLP/AI-ML classification + risk scoring.
- **Protection layer:** OpenCV-based blur/masking and protected output rendering.

5.3 CI/CD and fast delivery enabler

- Automatically run unit tests, linting, and basic integration checks on every change.
- Produce repeatable build artifacts for a staging/demo environment.
- Enable safe and frequent releases of incremental features such as OCR, new detection rules, and dashboard controls.

6 Evaluation Criterion

...measurable and attributable

We define success using measurable metrics tied directly to the real-time detection and censoring workflow.

6.1 Primary evaluation metric

Detection-to-censor latency (DCL): time between a sensitive region becoming available to the detection pipeline and the corresponding region being masked in protected output.

- **Target:** median DCL ≤ 1 second during the controlled pilot window.
- **Attribution:** measured using timestamps recorded at capture, OCR/detection, and masking stages.

6.2 Secondary evaluation metrics

- **Detection precision:** percentage of flagged regions that are actually sensitive.
- **Recall:** percentage of known sensitive items successfully detected and censored.
- **False-positive rate:** percentage of normal text incorrectly masked.
- **OCR accuracy:** correctness of extracted text on representative screen content.
- **Processing performance:** usable frame rate/latency on the target laptop environment.
- **Privacy:** raw screen frames and OCR text are not persistently stored by default.
- **User clarity:** user-reported confidence that sensitive information is protected, measured on a simple 1–5 scale.

6.3 Pilot validation plan

...high-level

- Prepare 20–30 controlled screen scenarios containing emails, phone numbers, identifiers, credentials/secrets, ordinary text, and mixed-content pages.
- Run each scenario through BlurGuard and compare detected/censored regions against the known sensitive regions.
- Measure detection precision, recall, false positives, OCR accuracy, and detection-to-censor latency.
- Conduct a small usability review with student users performing a screen-sharing task and record configuration and workflow feedback.

7 Scalability

...with foresight

The proposition should scale in both theoretical and practical senses.

1. Scalable (theoretically): the system should support more detection rules, larger screen regions, additional applications, and higher capture loads by decoupling capture, OCR, detection, and rendering components.

- Use configurable capture rates and region-of-interest processing to control CPU/GPU usage.
- Keep detection rules modular so new PII/secret patterns can be added independently.
- Use asynchronous processing where appropriate to prevent a slow OCR/detection step from freezing the user interface.

2. Operationally deployable (practically): the system should run as a desktop/local application with optional web-based controls or dashboard components, using common development and deployment

tooling.

8 Engine availability heuristic

A mature set of “engines” and libraries is available to implement BlurGuard as a practical privacy-focused system:

- Screen-capture libraries such as MSS/PyAutoGUI.
- OpenCV for preprocessing, coordinate handling, and blur/masking.
- Tesseract or EasyOCR for OCR and text bounding boxes.
- Python regex/rule engines and lightweight NLP/AI-ML components for sensitive-data classification.
- FastAPI for optional REST APIs and service separation.
- Streamlit or React for configuration/dashboard interfaces.
- Git/GitHub and CI tooling for version control, testing, and repeatable delivery.

We will use these mature components to focus on privacy workflow design, correctness, real-time performance, and measurement rather than inventing new infrastructure.

9 Project scope and deliverables

...iteration-friendly

9.1 Initial deliverable

...first iteration

- Screen capture for a selected screen/window/region.
- Image preprocessing and OCR-based text extraction.
- Basic sensitive-data detection using regex and predefined rules.
- Bounding-box based blur/masking of detected sensitive regions.
- Protected screen output for demonstration.
- Basic event logging and timestamps for latency measurement.
- CI: build + unit tests + linting for the core processing modules.

9.2 Subsequent deliverables

- AI/ML-assisted contextual PII classification and risk scoring.
- Detection of developer secrets such as API keys and tokens.
- Application/window awareness and configurable user rules.
- Real-time temporal tracking to reduce flicker and repeated detection.
- Dashboard for detection statistics, configuration, and performance monitoring.
- Advanced support for QR codes, sensitive documents, and clipboard-related privacy checks where feasible.
- Improved cross-platform packaging, security hardening, and deployment readiness.

10 Risks and mitigations

- **OCR errors:** use preprocessing, configurable thresholds, and multiple OCR strategies where necessary; evaluate accuracy on representative screens.
- **False positives/negatives:** combine structured rules with contextual classification and expose configurable sensitivity levels.
- **Performance overhead:** limit capture rate, process regions of interest, and use asynchronous

pipelines.

- **Privacy leakage:** keep processing local-first, minimize logs, avoid persistent raw-screen storage, and clearly separate diagnostic data from screen content.
- **User adoption:** provide a simple start/stop workflow, understandable alerts, and minimal configuration for common use cases.
- **Evaluation ambiguity:** define ground-truth test screens and instrument timestamps before development begins.
- **Operational issues:** test on representative laptops and maintain CI-backed repeatable builds.

11 Summary

This project proposes BlurGuard, a deployable privacy-focused system that detects and censors sensitive information during real-time screen sharing. It meets the course criteria by emphasizing time-to-value through rapid iterations, using mature OCR/computer-vision and AI/ML components, supporting CI/CD for safe incremental releases, and evaluating the system through measurable criteria such as detection-to-censor latency, precision, recall, false positives, OCR accuracy, and processing performance. The project remains focused on practical engineering, privacy, and scalability readiness rather than research-driven novelty.